

COSC 2030 QuickSort Assignment page 1

Goals of this assignment: See an implementation of QuickSort, review previous concepts of parallelization, learn about task parallelization, learn some new features of OpenMP.

Note: This assignment will involve the use of the OpenMP and parallelization, so don't use codeHS, you will need to be running on your local environment.

Note to on-line students(or anyone not in class): This was intended as a pair-programming, in-class assignment, so spend 2 hours on this and then turn in what you have and we will get together and discuss it so that you can see what the solution looks like.

1. In the quicksort.cpp file you will find some starter code that implements the quicksort algorithm. If you need a review on the quicksort algorithm take a moment to look up a quick video or other tutorial. Also take a moment to compile and run the code and make sure it is working properly.
2. We will be attempting to parallelize this code in this assignment and we want to be able to compare the parallel version to the serial version, [so add the elements from the <chrono> library](#) to measure the speed of the serial code.
3. The core of the quicksort algorithm is :

```
/* Base case: If there are 1 or zero elements to sort,
partition is already sorted */
if (low >= high) {
    return;
}

/* Partition the data within the array. */
pivot = partition(v, low, high); //returns sorted location of pivot
serialQuickSort(v, low, pivot-1); //recursively sort low partition
serialQuickSort(v, pivot+1, high); //recursively sort high partition
```

Could the two calls to 'serialQuickSort' happen at the same time, concurrently? Discuss this with your team before going to the next section.

[Go to next section](#)